

ReturnIQ

AI Return Intelligence Platform — Beginner's Guide to Your Own Project

A plain-language walkthrough of what's been built, why every decision was made, and how to answer questions about it in an interview.

Project: ReturnIQ — built for a fictional single-seller e-commerce platform called EliteMart

Purpose: Automate return-request review using an AI agent, and help customers trust products before buying

1. The Big Picture — What Is This App?

Imagine an online store (we're calling it **EliteMart**). Customers buy things, and sometimes they want to return a product. Right now, in most real companies, a human has to look at every single return request and decide: is this a genuine return, or something suspicious? That's slow and inconsistent.

ReturnIQ replaces that first manual check with an **AI agent** — a program powered by an AI model (Gemini) that investigates each return request itself, the way a smart employee would: checking the customer's history, looking at the product's past returns, and reading reviews — before deciding whether to approve, reject, or send the request for a human to look at.

There are two types of people who use the app:

- **Customer** — buys products, submits returns, tracks their status, and can chat with an AI assistant.
- **Seller** — this is EliteMart's own account (just one login representing the business), which reviews the AI's return decisions and can approve, reject, or override them.

Why call it an "AI Agent" and not just "using AI"?

Most simple AI features just take one question and give one answer — like asking ChatGPT a question. That's not what happens here for the return-checking part. Instead, the AI is given a set of tools (small functions) it can call — like a detective choosing which clues to check. It decides *on its own* which tools to use and in what order, based on what it's already found. That decision-making loop is what makes it an "agent" rather than a simple AI call.

Interview Q: What's the difference between calling an LLM once and building an agent?

Answer: A single LLM call takes one input and returns one output — no decisions about what to do next. An agent is given tools/functions it can choose to call, and it decides which ones to use and in what

sequence based on what it discovers along the way, before producing a final answer. My Return Intelligence Agent investigates step-by-step rather than following one fixed script.

2. The Database — Where Everything Is Stored

A database is just an organized set of tables, like several linked Excel sheets. We used **MySQL**, a very common type of database that stores data in rows and columns, and lets you link tables together.

ReturnIQ has 6 tables:

Table	What it stores	Why it exists
users	Login info for customers and the seller	So people can log in and the app knows their role
products	The product catalog + a Return Health Score	So customers can browse items, and the score shows how trustworthy
orders	Every purchase a customer made	A return can only happen if a real order exists — this proves it
returns	Return requests + the AI's findings	This is the core of the app — stores the request and the AI agent's
reviews	Customer feedback + star ratings	The AI reads these to spot patterns, like many people complaining
chat_history	Messages between customers and the AI assistant	So the chatbot remembers earlier messages in a conversation

Key design choices in the schema (and why)

- **role ENUM('customer','seller')** on the users table — a simple, fixed list of allowed values. This is how the app tells customers and the seller apart, and controls what each can access (this is called **Role-Based Access Control**, or RBAC).
- **return_health_score** on products (starts at 100, out of 100) — this number goes down as a product gets more returns/bad reviews. It's the single number that tells a customer "is this product reliable?" at a glance.
- **order_id is UNIQUE on the returns table** — this means one order can only ever have one return request linked to it. It stops someone submitting the same return twice.
- **The ai_* columns on returns** (ai_risk_score, ai_recommendation, ai_insight, ai_explanation, ai_confidence) start empty and get filled in by the AI agent after it investigates. Storing this (instead of just showing it once and forgetting it) means the seller can review the AI's reasoning later, and there's a record of every decision — like an audit trail.
- **Foreign keys** (e.g., orders.user_id → users.id) are like arrows connecting tables — they guarantee you can't have an order for a customer who doesn't exist, or a return for an order that doesn't exist. This keeps the data trustworthy.
- **Indexes** (like idx_orders_user) are shortcuts that make searching faster — without them, MySQL would check every single row to find matches; with them, it jumps straight to the right rows. Useful once the tables have thousands of rows, even though it's not obvious with only a few.

Interview Q: Why is `order_id` unique on the `returns` table but not on `orders`?

Answer: Because one order might never be returned at all — that's fine, no constraint needed there. But if a return does happen, it should only happen once per order — a customer can't request two separate returns for the same purchase. The `UNIQUE` constraint enforces that rule directly in the database, so it's not just checked in the app code, which could have a bug.

3. Authentication (Login System) — JWT + bcrypt

Authentication is how the app knows who you are when you log in, and keeps you logged in without asking for your password on every single click.

bcrypt — protecting passwords

When someone creates an account, their password is never stored as plain text (e.g., "mypassword123"). Instead, it's run through **bcrypt**, which scrambles it into a long, random-looking string (a "hash") that can't be reversed back into the original password. When someone logs in, bcrypt checks if their typed password, once hashed, matches the stored hash — without ever storing or seeing the real password again.

JWT — staying logged in

JWT stands for **JSON Web Token**. After you log in successfully, the server gives your browser a signed token (a long string of letters/numbers). Your browser sends this token with every future request, and the server checks the signature to confirm it's really you, without needing to check the database every single time.

This project uses **two tokens**:

- **Access Token** — short-lived, used for regular requests (like viewing your orders).
- **Refresh Token** — longer-lived, used only to get a new Access Token once the old one expires, so you don't have to log in again constantly.

They're signed with two *different* secret keys (`JWT_SECRET` and `JWT_REFRESH_SECRET`). This is a security choice: if one type of token's secret were ever leaked, the other token type is still safe.

Interview Q: Why use JWT instead of something like Firebase Authentication?

Answer: Firebase would have handled all of this for me automatically, but I chose to implement JWT myself because it means I actually understand and can explain how login sessions work — password hashing, token signing, expiry, and refresh — rather than relying on a service that hides those details. For a project meant to show my own engineering skill, that depth matters more than saving setup time.

Interview Q: What's the difference between the access token and refresh token?

Answer: The access token is short-lived and sent with every request to prove who I am — if it leaks, the damage window is small because it expires quickly. The refresh token lives longer and is only used to silently get a new access token, so the user doesn't have to log in repeatedly. Splitting them limits how much damage a leaked token can do.

4. The Backend — Node.js + Express

The backend is the part of the app that runs on a server (not in the user's browser). It handles requests like "log me in," "show me my orders," or "check this return request," talks to the database, calls the AI, and sends a response back.

Key libraries and why each was chosen

Library	Job	Why this one
express	Handles incoming web requests (routes)	Simple, extremely widely used, well documented
mysql2	Talks to the MySQL database	Supports async/await, modern and fast
jsonwebtoken	Creates and checks JWT login tokens	Standard library for JWT in Node.js
bcryptjs	Hashes passwords securely	Pure JavaScript — installs easily with no extra setup needed
dotenv	Loads secret settings from the .env file	Keeps secrets out of the actual code
cors	Lets the frontend (different address) talk to this backend	Browsers block cross-origin requests by default; this allows it safely
multer	Handles uploaded files (like return photos)	Needed before an image can be sent to Cloudinary
cloudinary	Stores uploaded images online	Free tier, simple API, reliable image hosting
@google/genai	Talks to Google's Gemini AI model	Official SDK — powers both the fraud agent and chatbot

Interview Q: Why bcryptjs instead of the original bcrypt library?

Answer: The original bcrypt library needs to compile native C++ code during installation, which can fail depending on someone's operating system or setup. bcryptjs is a pure JavaScript version — slightly slower, but installs anywhere with zero friction. For a project this size, that reliability trade-off is worth it.

5. Environment Files (.env) — Keeping Secrets Safe

The .env file holds real secret values: database password, JWT secret keys, and the Gemini API key. This file is **never uploaded to GitHub** — it's listed in a file called .gitignore, which tells Git to ignore it completely.

Alongside it, there's a .env.example file, which lists the same variable *names* but with no real values filled in. This gets committed to GitHub so that anyone (including future-you on a new laptop) knows exactly what secrets they need to provide, without ever exposing the real ones.

Interview Q: Why do you need both .env and .env.example?

Answer: .env holds my real, private secrets and is never committed to version control. .env.example is a template showing what variables the app expects, with placeholder values, so the setup process is documented without leaking anything sensitive. It's a standard practice for keeping credentials out of a public repository.

Important reminder: Always double check that `.env` is listed inside your `.gitignore` file before your first "git push" — accidentally committing real secrets to GitHub is a common and serious mistake, and if it happens, those credentials should be treated as compromised and changed immediately.

6. Line-by-Line Code Walkthrough (The Actual Logic)

This section takes real snippets from your files and explains exactly what each line does — this is the part to re-read before an interview, since it's the difference between saying "I used AI to build this" and saying "I understand what my code does."

6.1 — schema.sql: the users table

```
CREATE TABLE IF NOT EXISTS users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  display_name VARCHAR(255) NOT NULL,  
  role ENUM('customer', 'seller') NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

- **CREATE TABLE IF NOT EXISTS** — makes a new table, but if you accidentally run this script twice, it won't crash by trying to create a table that already exists.
- **id INT AUTO_INCREMENT PRIMARY KEY** — every row gets a unique number automatically (1, 2, 3...), and PRIMARY KEY means this column is the official unique identifier MySQL uses to tell rows apart.
- **email VARCHAR(255) NOT NULL UNIQUE** — a text field up to 255 characters. NOT NULL means this field can never be left empty. UNIQUE means no two users can share the same email — this is what stops duplicate account signups.
- **role ENUM('customer', 'seller') NOT NULL** — this column can *only* ever hold one of these two exact words. If your code tried to insert `role = 'admin'`, MySQL would reject it. This is a safety net at the database level, not just in your app's code.
- **password_hash VARCHAR(255)** — notice it's called `password_hash`, not `password`. This column never holds the real password, only the scrambled bcrypt output.
- **created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP** — you never have to manually set this; MySQL automatically stamps the exact moment the row was created.

6.2 — schema.sql: the returns table (the most important one)

```
order_id INT NOT NULL UNIQUE,  
...  
ai_risk_score INT DEFAULT NULL,  
ai_recommendation ENUM('approve', 'reject', 'manual_review') DEFAULT NULL,  
...  
FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
```

- **order_id INT NOT NULL UNIQUE** — links this return to exactly one order, and UNIQUE guarantees that same order can't be returned twice.
- **ai_risk_score ... DEFAULT NULL** — this column starts empty on purpose. It only gets filled in *after* your AI agent finishes investigating. NULL here basically means "not yet analyzed."
- **FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE** — this is a link (a promise) that order_id must match a real row in the orders table. ON DELETE CASCADE means if that order is ever deleted, MySQL automatically deletes this return too, instead of leaving orphaned, broken data behind.

6.3 — seed.sql: safely re-running your seed data

```
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE chat_history;
TRUNCATE TABLE reviews;
TRUNCATE TABLE returns;
TRUNCATE TABLE orders;
TRUNCATE TABLE products;
TRUNCATE TABLE users;
SET FOREIGN_KEY_CHECKS = 1;
```

Why this order matters: tables that *reference* other tables (like chat_history, which points to users) must be cleared **before** the tables they point to. If you tried to delete users first while chat_history still referenced them, MySQL would block it to protect data integrity. Temporarily switching FOREIGN_KEY_CHECKS off lets you clear everything in any order without those checks getting in the way, then it's switched back on immediately after so the safety rules apply again for normal use.

Interview Q: Why turn foreign key checks off, even briefly? Isn't that risky?

Answer: It's only turned off for the few seconds it takes to wipe and reseed test data on my own local machine — never in production. It avoids having to figure out the exact dependency order manually every time, and it's switched back on immediately afterward so the database keeps enforcing its normal safety rules for everything that happens next.

6.4 — seed.sql: realistic product data with a purpose

```
(9, 'Apex Run Men\'s Running Shoes', 'Footwear', 3999.00, ..., 58),
(1, 'SoundWave Max Wireless Headphones', 'Electronics', 4999.00, ..., 88),
```

Notice the last number in each row — that's the return_health_score. The running shoes start at 58 (low — lots of past problems), while the headphones start at 88 (high — mostly reliable). This isn't random: it's deliberately paired with matching seeded reviews (e.g., "too narrow," "caused blisters") so that when your AI agent reads the reviews table for the running shoes, it finds a real, consistent pattern to detect — instead of random unrelated complaints. This is what makes the AI's output look genuinely intelligent in a demo, rather than arbitrary.

Interview Q: How did you make sure your AI agent's output would look meaningful in a demo, not just random?

Answer: I deliberately designed the seed data so certain products have a consistent story across tables — for example, a pair of running shoes has both a low `return_health_score` and several reviews mentioning narrow sizing, plus return records citing the same reason. That way, when the agent reads return history and reviews together, it finds a real, coherent pattern to identify — proving the tool-calling logic actually works, not just that it produces plausible-sounding text.

6.5 — package.json: what each dependency line means

```
"dependencies": {  
  "bcryptjs": "^2.4.3",  
  "jsonwebtoken": "^9.0.0",  
  "@google/genai": "^2.10.0"  
}
```

- The `^` symbol in `"^2.4.3"` means "this version or any newer compatible version" (up to the next major version). It lets you get small bug-fix updates automatically without risking a version that breaks your code.
- **package-lock.json** (the file npm also generates) records the *exact* version that got installed on your machine, so if you or someone else runs `npm install` again later, they get the identical setup — this is what makes builds reproducible.

6.6 — .env: how secrets get loaded into your code

```
require('dotenv').config();  
const dbPassword = process.env.DB_PASSWORD;
```

The `dotenv` package reads your `.env` file when the server starts and copies each line into `process.env` — a special object Node.js gives every running program to hold environment settings. From then on, your code reads `process.env.DB_PASSWORD` instead of having the real password typed directly into a `.js` file. This is the mechanism that keeps secrets out of your source code.

7. What's Being Built Next: db.js (Database Connection Pool)

The next file connects the backend code to the actual MySQL database. Instead of opening a brand new connection every single time someone makes a request (slow, wasteful), it creates a **connection pool** — a small group of ready-to-use connections that get reused across many requests.

Interview Q: Why use a connection pool instead of a single connection?

Answer: A single connection can only handle one query at a time — if two people use the app simultaneously (or the AI agent makes several database calls back-to-back), requests would have to wait in line. A pool keeps multiple connections open and ready, so several queries can run at the same time without blocking each other.

8. How to Describe This Project in 60 Seconds

"I built ReturnIQ, a full-stack platform for an e-commerce store that automates return-request review using an AI agent. Instead of a human manually checking every return, my AI agent investigates each request by calling functions I wrote — checking the customer's return history, the order details, and whether the stated reason matches known product issues from reviews — and decides on its own which of these to check and in what order, before recommending approve, reject, or manual review, along with an explanation. I also built a customer-facing AI assistant chatbot, and the whole thing runs on a stack I own end-to-end: React frontend, Node/Express backend, MySQL database, and JWT-based authentication that I implemented myself rather than using a managed service, so I understand exactly how it works."

Keep this document updated as more files get built — treat it as your personal reference for explaining every part of ReturnIQ with confidence.